



Facultad de Ingeniería

Diseño de Aplicaciones 2

Obligatorio 2

Agustín Madrid – 270635

Joaquin Rodriguez – 231355

Juan Diego Meneses – 224304

[Link al repositorio](#)

2024

Índice:

Introducción	2
Descripción General del Trabajo	2
Errores y deuda técnica	3
Principales cambios con respecto al obligatorio anterior	3
Reorganización de los paquetes	3
La paginación pasa a estar en los repositorios	3
Las sesiones de los usuarios están persistidas en la base de datos	4
La utilización de Reflection	4
Vista Lógica	4
Vista de componentes	6
Vista de Procesos	7
Vista de despliegue	8
Vista de Casos de Uso	8
Crear una Home	8
Crear un Device	10
Modelo de tablas	12
Decisiones de Diseño más importantes	13
Cómo funcionan los roles y los permisos	13
Cómo funcionan los Home Devices	14
La creación de la clase MemberSettings y para que funciona	14
Manejo de Excepciones y Exceptionfilter	15
Hacer DeviceTypes Enum	16
Seed Data	16
Importers	16
Validators	17
Métricas de Diseño	17
Análisis de las métricas según los principios de paquetes	21
CPP: Principio de clausura común	21
CRP: Principio de reutilización	21
SAP: Principio de abstracción de paquetes.	21
SDP: Principio de Dependencias Estables.	22
Anexo	23
Anexo 1: Diagrama de clases de BussinessLogic	23
Anexo 2: Especificación de la API	24
Anexo 3: evidencia de Code Coverage	26

Introducción

Descripción General del Trabajo

Domoviz es una aplicación web que cuenta con una arquitectura cliente-servidor. Esta consta de dos grandes partes, el frontend implementado en Angular para esta entrega y una API implementada para la entrega anterior que fue modificada para esta entrega para tener nuevas funcionalidades y necesidades específicas del frontend. Nuestra aplicación tiene por objetivo centralizar y gestionar diferentes dispositivos inteligentes en un hogar. Los dispositivos inteligentes incluyen cámaras de seguridad, sensores de ventana, sensores de movimiento y lámparas inteligentes.

Nuestra plataforma permite que múltiples usuarios con diferentes roles, como administradores, dueños de empresas y dueños de hogares, puedan interactuar de manera eficiente y segura con los dispositivos. Cada rol tiene a su vez una lista de permisos los cuales especifican qué cosas puede hacer y cuáles no, dentro de la aplicación. De manera general los administradores gestionan todas las cuentas. Los dueños de empresas tienen su empresa específica donde pueden exponer los distintos dispositivos que quieren ofrecer. A su vez, los dueños de hogares pueden decidir qué productos añadir a su hogar, para poder tener diferentes controles que permiten mayor gestión a los smart devices. Además los dueños de hogares gestionan los permisos de los diferentes miembros pertenecientes al hogar y pueden recibir notificaciones en caso de que alguno de sus dispositivos vinculados se active. Gracias a esta implementación con los permisos el requisito de que los usuarios con roles de administrador y dueño de empresa puedan actuar como dueños de Hogar fue muy sencilla ya que solo le agregamos los permisos que son específicos de HomeOwner.

Por otro lado los dueños de empresa pueden incluir sus propios modelos validadores para la creación de diferentes dispositivos y facilitar así la gestión de su principal fuente de negocio como son los dispositivos.

Además, incluimos un importador que sirve como carga masiva para hacer más amena la creación de los diferentes productos que la empresa quiere manejar.

Manejamos las Features usando GitFlow en cada rama se implementa un Fix o una Feature nueva y se y se hace una pull Request a develop.

Dejando de lado las funcionalidades, para organizar el trabajo creamos un tablero de JIRA con tickets y cada uno de los integrantes del equipo fueron asignándose tarjetas a medida que implementamos la aplicación. Este tablero consistía de 4 columnas: To Do, In Progress, In Review y Done.

Por lo general siempre que se ponía In Review un ticket era para que un compañero revisará de manera detallada el código. Utilizamos el code Analysis proporcionado en clase y nombres de clases y métodos explícitos para mantener el clean Code en todo momento.

Errores y deuda técnica

Por falta de tiempo para la entrega decidimos dejar el `IDeviceImporter` como está ahora en el cual el nombre del file y la ruta están hardcoded y no se necesita de recibir ningún parámetro extra sabemos que este no siempre es el caso y que debería de haber una función en el `IDeviceImporter` que sea `GetParams` para que esta reciba todos los parámetros necesarios para la implementación. Otro deuda técnica que tenemos es nuestro log out por falta de tiempo nuestro logout te redirige al login borra el local Storage. No borra realmente la sesión de la base de datos. Esto lo hicimos de esta manera ya que en el hipotético caso de que tengas en 2 computadores abiertas la misma sesión y la cierres en una de ellas no te debería de cerrar las otras.

Como última cosa encontramos que nuestro Get de devices no tiene pagination en el front por lo cual aunque si está implementado en el back no nos dimos cuenta de hacerlo en el front hasta último momento.

Principales cambios con respecto al obligatorio anterior

Reorganización de los paquetes

Después de la primera corrección entendimos que la separación en tantos paquetes complejizaba la solución, además de generar métricas inexactas y comportamientos indeseados en términos de diseño. Este cambio sigue el **Principio de las Clases que Cambian Juntas Deben Estar Juntas** (*Common Closure Principle, CCP*).

Nuestras clases de **IBusinessLogic**, **BusinessLogic**, **Domain**, **IDataAccess** cambian siempre juntas por lo cual las juntamos todas en **BusinessLogic**. Nuestras clases de Models fueron a parar en el paquete WebAPI porque en realidad tendría que ir allí. Suprimimos el paquete ServiceFactory ya que nos dimos que es responsabilidad de WebAPI ahora esa funcionalidad se encuentra en el `program.cs`. Veremos más adelante, cómo afecta a las métricas de paquetes este cambio en la sección de métricas de diseño ya que consideramos un gran progreso ver esta diferencia.

La paginación pasa a estar en los repositorios

Uno de los problemas más significativos que tuvimos en nuestra solución anterior fue realizar la paginación y el filtrado directamente en los controladores. Este enfoque resultaba en una pérdida considerable de calidad en las respuestas y de eficiencia general del sistema, especialmente en posibles escenarios futuros con grandes volúmenes de datos.

Impacto del cambio

Obtuvimos mejora en el rendimiento, dado que solo recuperamos los datos necesarios de la base de datos. Por ende disminuimos el uso de memoria y el tiempo de gestión. Además evitamos transferir grandes volúmenes de datos, por lo cual reducimos el tráfico de datos.

Las sesiones de los usuarios están persistidas en la base de datos

En la iteración anterior, las sesiones de los usuarios no eran guardadas en la base de datos. Este diseño presentaba varios inconvenientes, especialmente al considerar la naturaleza distribuida de muchas aplicaciones modernas y la necesidad de escalabilidad. Además, rompía con el principio de stateless establecido en el enunciado, donde cada solicitud HTTP debería ser independiente de cualquier estado mantenido en memoria por el servidor.

Por ejemplo, en un escenario donde nuestra aplicación estuviera desplegada en varias instancias (por balanceo de carga o alta disponibilidad), una solicitud de un usuario que ya se hubiera autenticado en una instancia podría ser redirigida a otra instancia. Si la sesión no estaba centralizada y persistida, el usuario tendría que autenticarse nuevamente, lo que afectaría negativamente la experiencia del usuario y la funcionalidad del sistema.

Impacto del cambio

Al persistir las sesiones en la base de datos, no solo mejoramos la arquitectura y el cumplimiento de los principios establecidos, sino que también preparamos el sistema para manejar mayores cargas y escenarios distribuidos de manera eficiente.

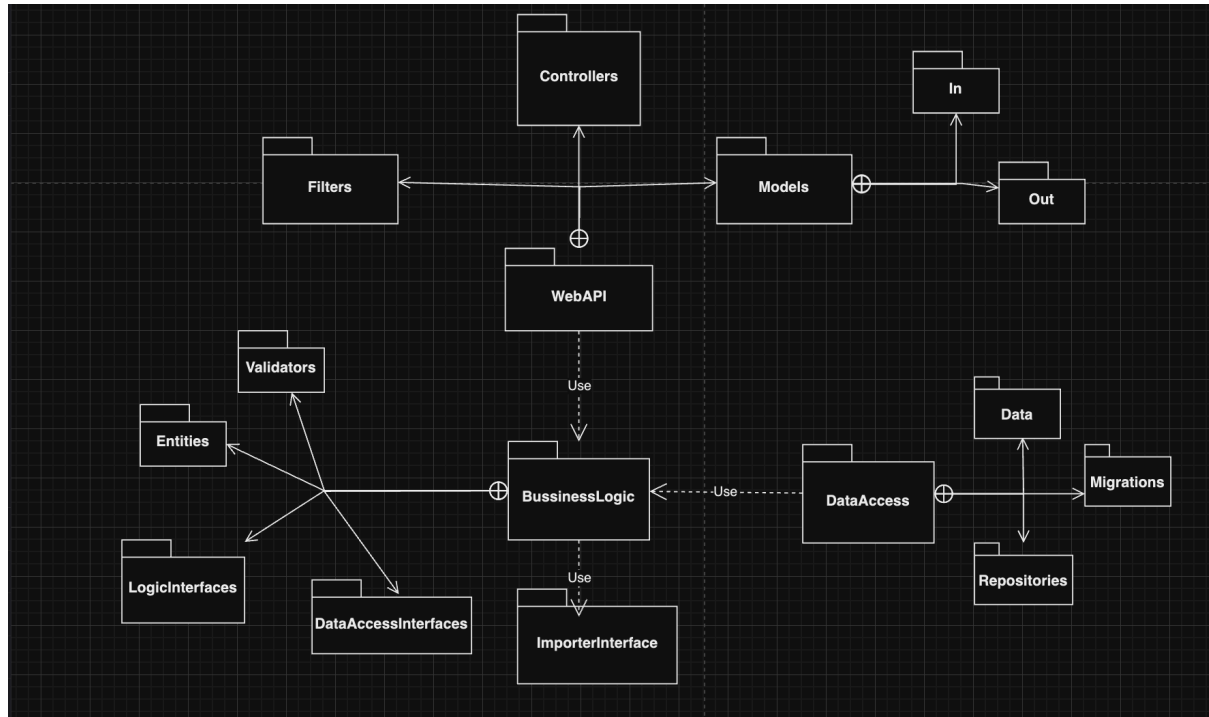
La utilización de Reflection

Nuestra aplicación utiliza la técnica de reflection para que ahora los usuarios puedan extender la aplicación ellos mismos. Esto lo pueden hacer mediante la implementación de nuevos métodos de validación para dispositivos o directamente al importar una gran cantidad de dispositivos a la vez en lugar de insertarlos manualmente.

Básicamente hay dos casos de uso para la reflection. La primera, para agregar modelos validadores, debe añadir un .dll a la carpeta “Validadores” ubicada en la WebApi. Esto le permite en tiempo real, agregar modelos validadores y seleccionarlos en la interfaz gráfica para una compañía en concreto. Por otro lado, utilizamos reflection para agregar importadores de devices. Esto permite al usuario generar una especie de carga masiva de dispositivos para el método que éste desee.

Vista Lógica

Lo primero que se nota a la vista es nuestro diagrama de paquetes que a diferencia del anterior tiene varios subpaquetes.



Primera capa

Nuestra primera capa. Nuestro paquete WebAPI es el que se encarga de la comunicación con el cliente, es la capa más alta de nuestra aplicación. Este tiene tres subpaquetes: el de Filters este contiene los tres filtros que de la aplicación; el de Controllers el cual contiene todos los controladores y el de Models el cual usamos para crear modelos de las entidades para no pasar clases del dominio estos en dos paquetes uno de salida y otro de entrada.

Segunda capa

Nuestra segunda capa, la capa de modelo de negocio, BusinessLogic este contiene todas las clases necesarias para implementar la lógica específica del negocio está contienen las implementaciones. Este tiene cinco subpaquetes el más importante es el de Entities el cual es el dominio de nuestra aplicación. Luego tenemos Validators en el cual tiene la implementación de los validadores de Dispositivos; La de LogicInterfaces y la de DataAccessInterfaces tiene las interfaces de la lógica y las de los repositorios respectivamente. En esta capa también tenemos un paquete de IImporter el cual usamos para la importación de Devices.

Tercera capa

Luego tenemos nuestra última capa de DataAccess el cual tenemos todas las funciones para acceder en la base de datos, estas clases las tratamos de hacer las más sencillas posibles que todo la responsabilidad de chequeos recaiga en la capa anterior. Además tenemos un paquete con todas las migraciones de la base de Datos y el paquete Data en el cual tenemos nuestro contexto.

Ahora mostraremos más en detalle el paquete más extenso que es de la BusinessLogic [Anexo 1].

En esta parte vemos cómo se conectan las clases de la businessLogic vemos que los las Logic implementan las ILogics y usan las IRepository que gracias a la inyección de dependencias se transforman la implementación de los repositorios.

Con respecto a la iteración pasada del obligatorio las nuevas adiciones dentro del modelo de negocio es la clase Room que la usamos para poder separar por cuartos una Home si se quisiera. Además agregamos un property name en Homes y HomeDevices para que se puedan poner alias para estas entidades, para cumplir los nuevos requerimientos.

Vista de componentes



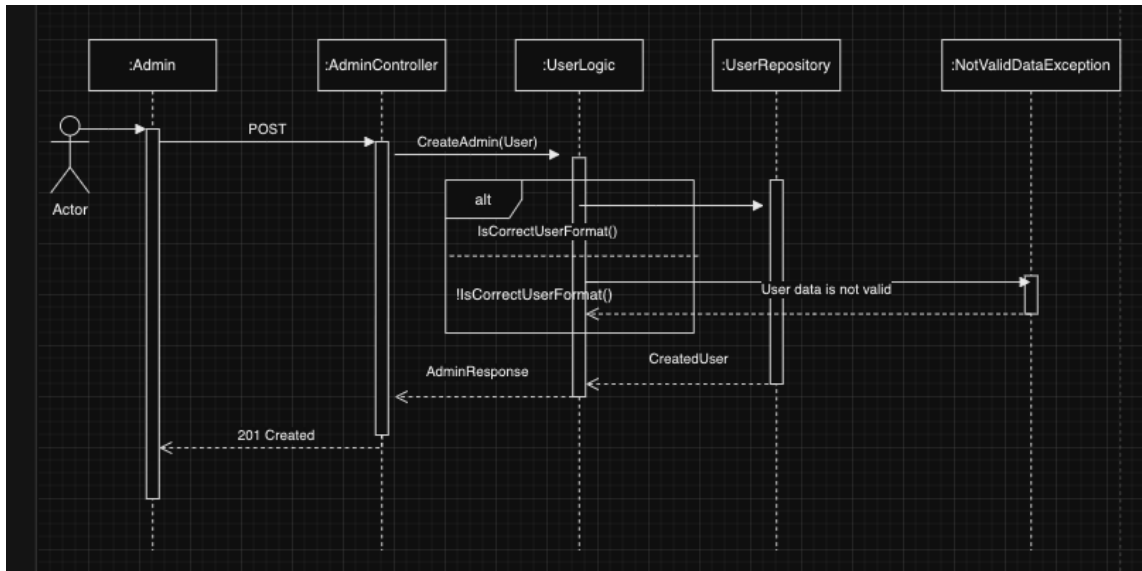
Aca podemos observar el diagrama de componentes en este podemos ver las tres capas claramente y que hay sólo dependencias entre capas mediante interfaces. Con esta arquitectura tenemos una alta cohesión ya que cada componente tiene una responsabilidad única. Esto permite también como gran ventaja hacer un TDD Top-Down mockeando los componentes de más bajo nivel.

Vista de Procesos

Ahora mostraremos algunos diagramas de secuencias que ejemplifican algunas de las features más importantes.

Creación de admin

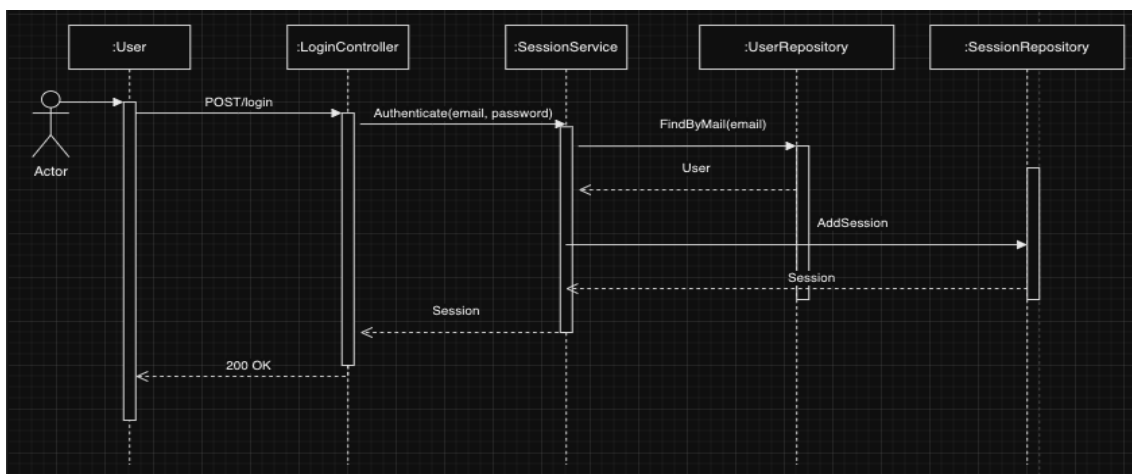
Acá mostramos como un admin crea un nuevo usuario:



En estos casos podemos apreciar como al efectuar una acción la información pasa por los 3 capas.

Login

Aquí mostramos como un usuario se loguea a su cuenta y se queda persistida su sesión en la base de datos.



Vista de despliegue

En este diagrama de despliegue podemos ver como tenes los 3 nodos y por que protocolo se conectan



Vista de Casos de Uso

Crear una Home

2. Actores:

- Usuario autenticado: Un usuario que ya tiene acceso al sistema después de haber iniciado sesión.

3. Propósito:

Permitir a los usuarios crear un nuevo registro de Home en el sistema con los datos necesarios.

4. Precondiciones:

- El usuario debe estar autenticado en el sistema.
- El sistema debe presentar acceso al formulario para crear un Home.

5. Flujo principal (camino feliz):

1. El usuario abre la aplicación y accede al Login.
 - Introduce el correo electrónico y la contraseña en los campos correspondientes.
 - Presiona el botón Submit.
 - El sistema valida las credenciales y permite el acceso al usuario (redirección al Dashboard principal).
 2. En el Dashboard, el usuario selecciona la opción "New Homes" del menú lateral izquierdo (aside menu).
 3. El sistema redirige al formulario "Create a new home".
 4. El usuario rellena los siguientes campos en el formulario:
 - Name: Nombre de la Home (por ejemplo, "Casa Central").
 - Location: Ubicación (por ejemplo, "Montevideo").
 - Latitude: Coordenada geográfica de latitud (por ejemplo, -34.9011).
 - Longitude: Coordenada geográfica de longitud (por ejemplo, -56.1645).
 - Member Count: Número de miembros que residen en esta Home (por ejemplo, 4).
 5. El usuario presiona el botón "Submit".
 6. El sistema valida la información:
 - Comprueba que todos los campos requeridos están completos y son válidos.
 - Verifica si ya existe una Home con el mismo nombre para evitar duplicados.
 7. Si todo es correcto:
 - El sistema registra la Home en la base de datos.
 - Muestra un mensaje de éxito: "Home created successfully".
 - Redirige al usuario a una vista donde puede gestionar las Homes existentes o volver al Dashboard.
-

6. Flujo alternativo:

[A1] Error de validación:

1. Si alguno de los campos del formulario está incompleto o tiene datos inválidos:
 - El sistema muestra un mensaje de error debajo del campo correspondiente (por ejemplo, "Latitude is required").
 - Permite al usuario corregir los datos.

[A2] Error en el login:

1. Si las credenciales son incorrectas:
 - El sistema muestra el mensaje "Invalid email or password".
-

7. Poscondiciones:

- El registro de la nueva Home queda almacenado correctamente en el sistema.
- El usuario puede ver la Home recién creada en la lista de Homes.

Crear un Device

2. Actores:

- **Company Owner:** Un usuario autenticado con rol de dueño de empresa.

3. Propósito:

Permitir a un **Company Owner** agregar un nuevo dispositivo asociado a su empresa.

4. Precondiciones:

- El usuario debe estar autenticado con una cuenta de tipo **Company Owner**.
- El usuario debe tener una **Company** previamente asociada. Si no, debe crear una antes de agregar un dispositivo.

5. Flujo principal (camino feliz):

1. El **Company Owner** abre la aplicación y accede al **Login**:
 - Introduce el **correo electrónico** y la **contraseña**.
 - Presiona el botón **Login**.
 - El sistema valida las credenciales y permite el acceso (redirección al Dashboard principal).
2. En el Dashboard, el usuario selecciona la opción "**Company**" del menú lateral izquierdo (**aside menu**).
3. El sistema verifica si el usuario ya tiene una **Company** asociada:
 - **[Condición: Tiene una Company]:**
 - Aparece el detalle de la empresa, con un botón llamado "**Devices**".
 - El usuario selecciona este botón y accede a la lista de dispositivos asociados a la empresa.
 - **[Condición: No tiene una Company]:**
 - Se muestra un botón llamado "**Create**".
 - El usuario selecciona este botón, rellena los datos requeridos (nombre de la empresa, etc.) y presiona **Create**.
 - El sistema guarda la nueva Company y redirige a la sección de dispositivos.
4. En la vista **Devices**, el usuario puede ver una lista de los dispositivos asociados a la empresa:
 - Si no hay dispositivos, la lista estará vacía.
5. El usuario selecciona el botón "**Add Device**".
6. El sistema redirige al formulario "**Create a new Device**".
7. El usuario completa los siguientes campos en el formulario:
 - **Name:** Nombre del dispositivo (por ejemplo, "Sensor de Movimiento").
 - **Type:** Tipo de dispositivo (por ejemplo, "MovementSensor").
 - **Model:** Modelo del dispositivo (por ejemplo, "SML024").
 - **Description:** Breve descripción (por ejemplo, "Sensor para detectar movimiento en interiores").
 - **Photo:** URL de la foto o nombre de archivo (string, formato .jpeg o .png).

8. El usuario presiona el botón "**Create Device**".
 9. El sistema valida los datos ingresados:
 - Comprueba que todos los campos están completos y son válidos.
 - Verifica que el tipo y formato de foto sean correctos.
 10. Si todo es correcto:
 - El sistema guarda el dispositivo en la base de datos.
-

6. Flujo alternativo:

[A1] Error en la validación de campos:

1. Si alguno de los campos está vacío o contiene datos inválidos:
 - El sistema muestra un mensaje de error debajo del campo correspondiente (por ejemplo, "Photo must be a .jpeg or .png file").
 - Permite al usuario corregir los datos y reintentar.

[A2] El usuario no tiene una **Company**:

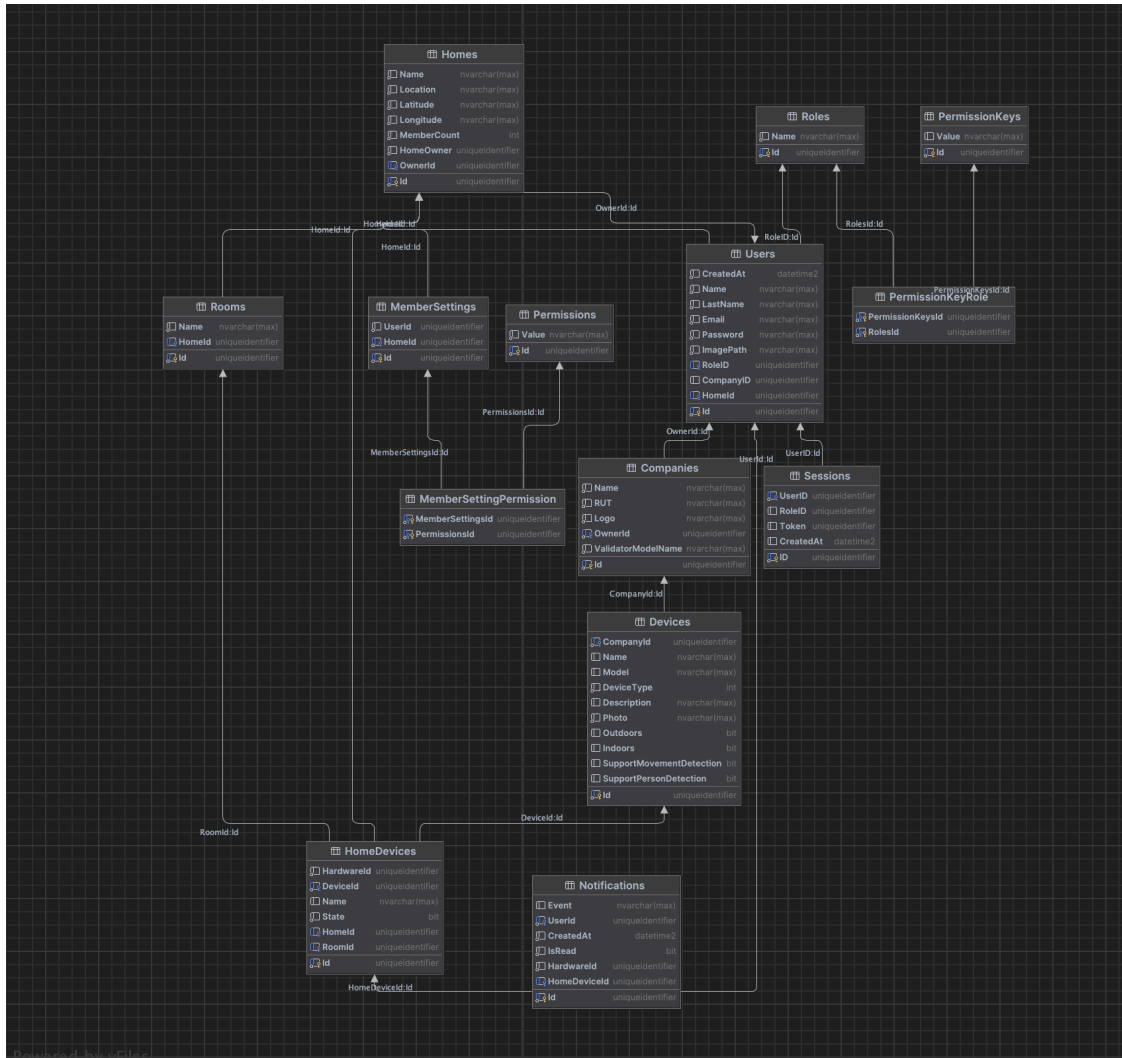
1. Si el usuario no crea una **Company**, no podrá acceder a la vista de dispositivos.
 - El sistema muestra un mensaje: "**You must create a company to add devices.**".
-

7. Poscondiciones:

- El nuevo dispositivo queda registrado correctamente en la base de datos y asociado a la **Company** del usuario.
- El dispositivo aparece en la lista de dispositivos de la empresa.

Modelo de tablas

Mostraremos ahora cómo quedó el modelo de tablas de nuestra aplicación generado por **Entity Framework**. A diferencia de la entrega tenemos **Rooms** y además tenemos las sesiones persistidas en la base de datos.



Cabe aclarar que en la creación del context además de insertar los datos de la Seed Data se especifican ciertos permisos. Como es el caso de las Homes que vimos la necesidad de agregar tres permisos. Siendo estos *CanAssociateDeviceToRoom*, *CanChangeHomeName*, *CanChangeHomeDevices*. Estos se implementaron con el fin de cumplir las nuevas funcionalidades.

Decisiones de Diseño más importantes

Cómo funcionan los roles y los permisos

Para empezar a explicar como funcionan los roles y los permisos, debemos hacer una pequeña introducción a los elementos que usamos para diseñar esta solución.

Por un lado tenemos a los Roles o como está definido en nuestro dominio : **Role**.

Esta clase es la encargada de tener el rol, su nombre, y los **PermissionKey** correspondientes a cada rol. Podemos ver que dentro de rol, esta lista de **PermissionKeys** es virtual. Cuando una propiedad de navegación se declara como virtual, **EF Core** puede interceptar las llamadas a esa propiedad. Esto permite que la propiedad se cargue automáticamente desde la base de datos en el momento en que se accede, en lugar de cargarse junto con la entidad principal.

Por otro lado **PermissionKey** es una clase que almacena los diferentes permisos para cada rol.

Este conjunto de ideas, nos permitió que a la hora de utilizar filtro de autorización, podamos limitar las funcionalidades a las que cada rol puede acceder.

Veamos un ejemplo, si un Admin quiere acceder a los datos de los usuarios. Primero se chequea que esté logueado. Luego se chequea el rol con su nombre y se busca en sus permisos el permiso que el controller le está pidiendo, en este caso por ejemplo sería *"CanManageUsers"*.

Ventajas de Utilizar Roles y Permisos

Control Granular de Acceso: Permite definir con precisión qué acciones puede realizar cada rol dentro del sistema. Esto asegura que los usuarios solo tengan acceso a las funcionalidades necesarias para su trabajo, mejorando la seguridad.

Facilidad de Mantenimiento: Al centralizar los permisos en la clase **PermissionKey**, es más fácil realizar cambios. Si se necesita modificar un permiso o agregar uno nuevo, solo es necesario actualizar la clase **PermissionKey** o la lista de permisos en el rol correspondiente, sin tener que modificar la lógica de acceso en múltiples partes del código.

Escalabilidad: Este enfoque permite escalar el sistema fácilmente. A medida que se añaden nuevos roles o permisos, se pueden gestionar sin complicaciones adicionales. Por ejemplo, si se introduce un nuevo rol "Editor", simplemente se pueden definir los permisos correspondientes sin alterar la estructura existente.

El enfoque utilizado, nos permitió incluso, ahorrarnos tiempo significativo gracias a que los usuarios con permisos extra cuentan también con los permisos de roles de mas bajo nivel. Permitiendo que el admin, también pueda crear homes así como el dueño de compañía.

Este tiempo ahorrado fue importante para enfocarnos en el resto de funcionalidades. Por ende vemos importante explicar que hacerlo escalable fue primordial para el rendimiento del equipo.

Auditoría y Seguimiento: Al tener roles y permisos bien definidos, es más sencillo llevar a cabo auditorías de seguridad y seguimiento de quién accede a qué funcionalidades. Esto es fundamental para mantener la conformidad y detectar cualquier acceso no autorizado.

Cómo funcionan los Home Devices

Los **HomeDevices**, son una clase intermedia que se genera al relacionar un Device con una Home. Básicamente al agregar un dispositivo a una casa, creamos un **HomeDevice** que nos viene con hardware id y otras características pedidas específicamente para la asignación de un dispositivo a un hogar.

Creación de HomeDevices

Cuando un usuario decide agregar un dispositivo a una casa, se genera automáticamente un objeto de tipo **HomeDevice**. Este objeto actúa como un enlace entre el dispositivo y el hogar, asegurando que cada dispositivo esté correctamente asociado a un entorno específico. Durante este proceso de creación, el **HomeDevice** se configura con varias características importantes:

Hardware ID: Este identificador único permite rastrear y gestionar el dispositivo en la red del hogar. Es esencial para realizar operaciones como la activación, desactivación o monitoreo del estado del dispositivo.

Estado: La propiedad de estado indica si el dispositivo está activo, inactivo o en algún otro estado relevante. Esto permite al sistema y a los usuarios tener una visibilidad clara sobre el funcionamiento de los dispositivos en tiempo real.

Configuraciones Específicas: Además del hardware ID y el estado, el **HomeDevice** puede incluir otras propiedades que se ajusten a las necesidades específicas del dispositivo. Por ejemplo, puede haber propiedades para la configuración de red, niveles de batería, o características personalizadas según el tipo de dispositivo (como temperatura para termostatos o niveles de luz para bombillas inteligentes).

Ventajas de Usar HomeDevices

La implementación de la clase HomeDevice ofrece varias ventajas:

Flexibilidad: Permite una gestión más flexible de los dispositivos dentro de un hogar, ya que cada dispositivo puede ser configurado de manera individual y asociado a las necesidades específicas del hogar.

Facilidad de Mantenimiento: Al tener una clase intermedia, el mantenimiento y la actualización de la información relacionada con los dispositivos se simplifica, facilitando tareas como la implementación de nuevos dispositivos o la modificación de los existentes.

Escalabilidad: La estructura permite escalar fácilmente el sistema a medida que se agregan más dispositivos o hogares, sin necesidad de complicar las relaciones entre las entidades.

La creación de la clase MemberSettings y para que funciona

La clase **MemberSettings** se creó para gestionar y personalizar las configuraciones de permisos de cada miembro dentro de un hogar. Su propósito es análogo al de los **Roles** con sus respectivas

PermissionKeys, pero con un enfoque más centrado en las preferencias individuales de cada miembro.

Estructura de MemberSettings

Propiedades del Miembro: La clase **MemberSettings** contiene propiedades que representan las configuraciones específicas de un miembro. Esto puede incluir información sobre las preferencias de notificación, acceso a dispositivos y otros permisos relevantes.

Lista Virtual de Permissions: Al igual que en la clase Role, en MemberSettings hay una lista virtual de Permission. Esta propiedad permite que EF Core cargue automáticamente las claves de permisos asociadas al miembro desde la base de datos cuando se accede a ella. Esto proporciona flexibilidad y eficiencia al gestionar las configuraciones.

Funcionalidad de MemberSettings

La funcionalidad de MemberSettings se centra en permitir a cada owner personalizar su experiencia y establecer qué permisos desean activar. Algunas de sus características incluyen:

Permisos Personalizados: A través de la lista de Permissions, cada owner puede seleccionar permisos específicos que deseen habilitar o deshabilitar. Esto es particularmente útil en un entorno donde diferentes miembros pueden tener diferentes necesidades y niveles de acceso.

Preferencias de Notificación: La clase puede incluir configuraciones relacionadas con cómo y cuándo los miembros desean recibir notificaciones. Esto permite una gestión más centrada en el usuario, donde cada miembro puede ajustar sus preferencias en cuanto a las notificaciones.

Seguridad y Privacidad: Al permitir que cada miembro defina sus propios permisos, se mejora la seguridad y la privacidad dentro del hogar. Cada miembro tiene el control sobre qué información y funcionalidades son accesibles para ellos, lo que reduce el riesgo de acceso no autorizado.

Manejo de Excepciones y Exceptionfilter

En nuestro sistema, el manejo de excepciones se gestiona mediante un ExceptionFilter global que intercepta y procesa todas las excepciones lanzadas durante la ejecución de las peticiones. Este filtro no solo mejora la consistencia de las respuestas en caso de errores, sino que también permite una gestión centralizada de las excepciones, lo que genera que tengamos un código más controlado.

Excepciones Customizadas

Para lograr este control centralizado, utilizamos excepciones customizadas (o personalizadas). Estas excepciones son específicas del dominio de la aplicación y permiten manejar errores con mayor precisión, en lugar de depender de las excepciones genéricas del sistema. Cada excepción customizada representa un tipo particular de error, lo que facilita su identificación y tratamiento.

Funcionamiento del ExceptionFilter Global

El ExceptionFilter global se encarga de "atrapar" o cachear las excepciones que se lanzan durante el procesamiento de las solicitudes. Dependiendo del tipo de excepción que cachea, el filtro devuelve

diferentes códigos de respuesta HTTP, permitiendo al cliente saber qué tipo de error ocurrió. Los códigos de respuesta que generalmente utilizamos incluyen:

- **400 Bad Request:** Para errores de validación u otros problemas con la solicitud enviada por el cliente.
- **409 Conflict:** Para situaciones en las que hay un conflicto con el estado actual del recurso (por ejemplo, intentos de crear un recurso que ya existe).

Los otros tipos de errores son lanzados directamente por el Authorization filter y el Authentication Filter que dan los códigos de respuesta 403 Forbidden o 401 unauthenticated.

Hacer DeviceTypes Enum

Con el fin de optimizar el desarrollo y ahorrar tiempo, hemos decidido utilizar un enum llamado **DeviceTypes** para representar los diferentes tipos de dispositivos en nuestro sistema. Por el momento, este enum contiene dos tipos distintos: WindowSensor, Camera, MovementSensor y SmartLamp .

Este enfoque no solo simplifica la gestión de los tipos de dispositivos, sino que también ofrece una manera clara y eficiente de discriminar cómo se guardan estos dispositivos en la base de datos.

Seed Data

Nuestro SeedData lo creamos en el context en el paquete Repositories.Data. En este al crear la base de datos Inserta todos los roles con sus respectivos permisos. Además de un primer administrador el cual tiene como email: admin@admin.com y su contraseña es admin.

En nuestra primera iteración de la aplicación manteníamos siete permisos estos no han cambiado. Los permisos son *CanCreateAdmin*, *CanCreateCompanyOwner*, *CanCreateCompany*, *CanManageUsers*, *CanGetCompanies*, *CanCreateADevice* y *CanGetCompanies*. Además como mencionamos anteriormente, se agregaron tres permisos extra, para poder cumplir con los requerimientos de esta parte de la entrega

Importers

Para facilitar la importación de dispositivos para las compañías se solicitó un importador en tiempo real y que para esta específica instancia, corriera un JSON importer.

La solución a la que llegamos, se acerca casi en su totalidad a lo solicitado. El usuario puede importar la cantidad de dispositivos que necesite desde un JSON o desde cualquier importador que desee implementar a la solución.

Para agregar un importer, el usuario programador debe generar un XImporter. Por ejemplo, un XMLImporter y agregar dentro, la lógica que vea conveniente para crear el importador, además de la ruta donde colocará cada .xml. De este importer, debe generar el .dll, y colocarlo en el path del input. Para que a la hora de hacer la importación, acceda al dll y haga el importe correspondiente.

Validators

Nuestra solución ahora admite adición de lógica ya implementada gracias a reflection esto se hace en concreto con los métodos de validación del nombre de los dispositivos. Nosotros en concreto creamos los 4 validadores pedidos en la letra pero en caso de querer nuevos se puede dejar un ddl en la carpeta Validators del proyecto con la implementación de la interfaz. Y la aplicación al momento de crear una empresa mostrará esa forma de validar modelos como una opción posible (de misma manera que si borramos los ddl de los formatos estos dejarán de estar como opciones).

Métricas de Diseño

La definición que tenemos de métrica es una medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo dado. Con la herramienta nDepend sacamos las métricas de nuestro paquete. Nos centraremos en la cohesión, inestabilidad, abstracción. Además veremos la gráfica de dolor. Pero antes de todo esto veamos qué nos dicen estas métricas.

Cohesión: La cohesión relacional mide qué tan relacionadas están entre sí están las clases de un mismo paquete. Se calcula como $(\text{el número de relaciones clases internas al paquete}) + 1 / (\text{el número total de de clases e interfaces en el paquete})$. Se entiende como buena entre 1.5 y 4.

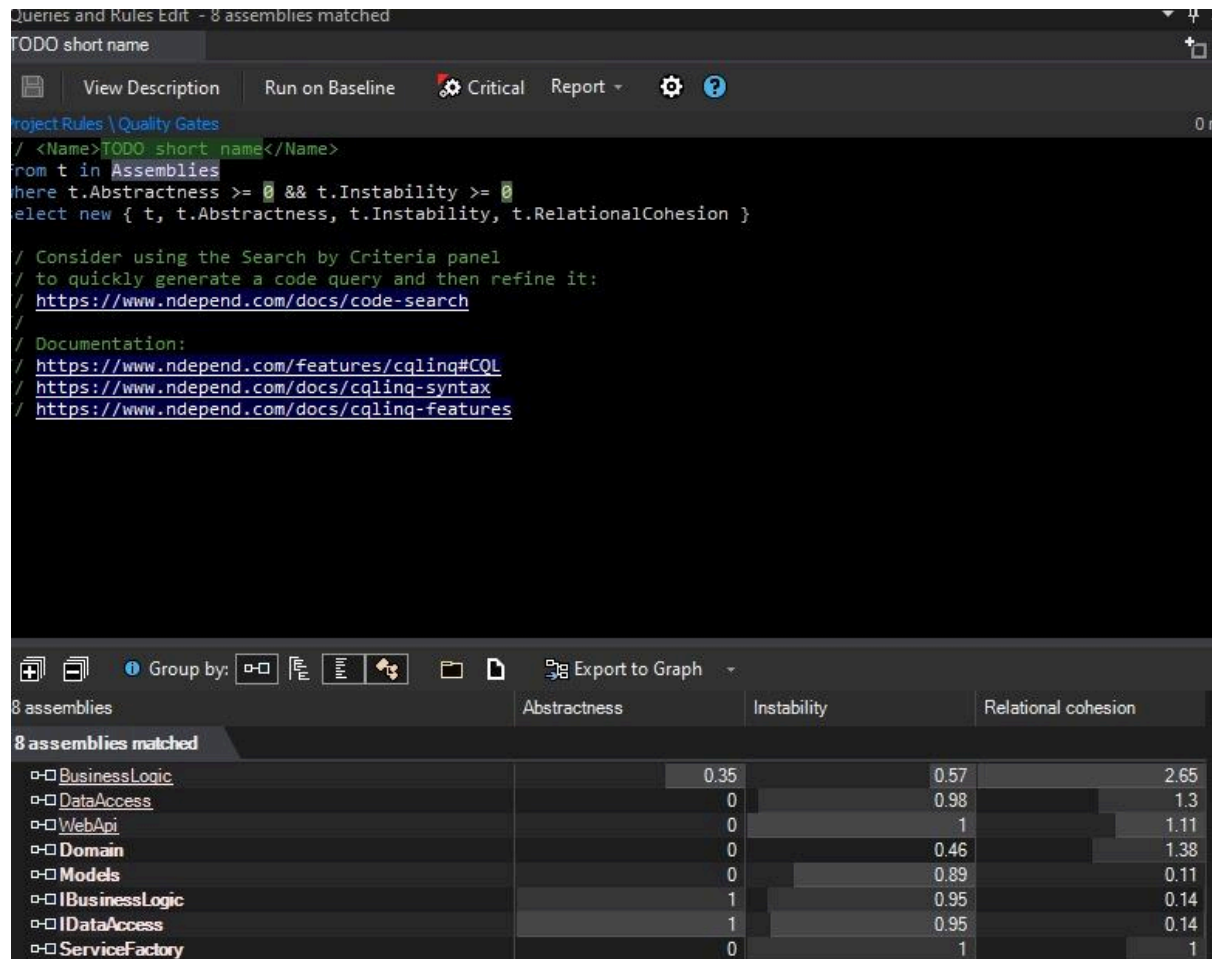
Inestabilidad: La inestabilidad mide cuán propenso es un paquete a cambiar debido a modificaciones en otros componentes. En otras palabras, nos dice qué tan estable es un componente frente a los cambios en el sistema. Se calcula como $(\text{Número de clases fuera del paquete que dependen de clases dentro del paquete}) / (\text{Número de clases fuera del paquete que dependen de clases dentro del paquete}) + (\text{Número de clases fuera del paquete de las cuales dependen clases dentro del paquete})$. Esta métrica es muy importante para el principio SDP (Stable dependencies Principle) el cual indica que la dirección de las dependencias debe de ir en dirección de la estabilidad.

Abstracción: La abstracción nos indica cuán abstracto es un paquete. Es decir que tanto se enfoca el paquete en conceptos generales y no en detalles de la implementación. Esta métrica también es muy importante en el principio SAP (Stable Abstractions Principle) el cual indica que los paquetes más estables también deben de ser los más abstractos.

Además gracias a nDepend nos da la Distancia de la secuencia principal con esto podemos ver que tan balanceado está nuestro paquete entre tan abstracto como es y qué tan inestable y concreto es. Esta gráfica nos indica 2 zonas en las que los paquetes no deberían estar: la zona de dolor y la zona de poca utilidad.

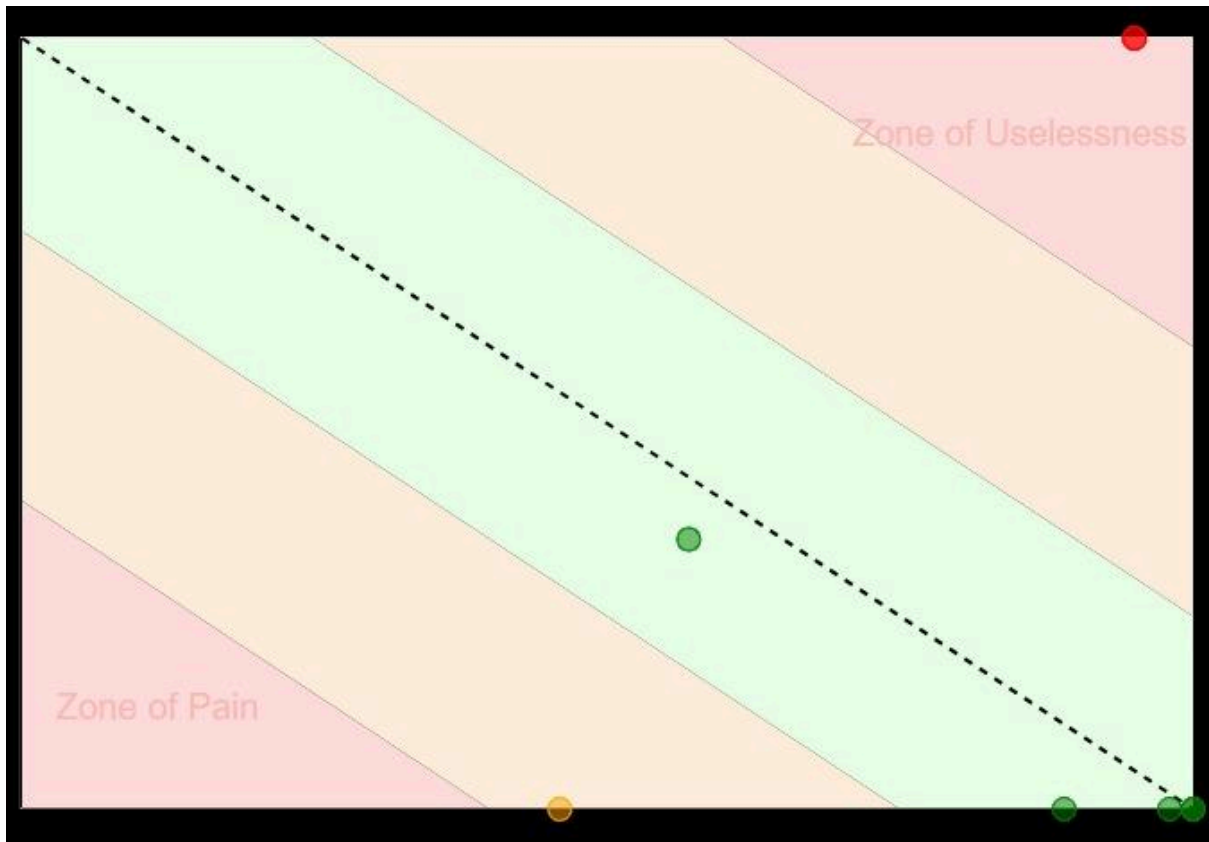
Primero encontramos interesante mostrar cómo nuestras métricas de paquetes mejoraron drásticamente gracias a la reorganización de los paquetes.

Métricas de la primera entrega:



Lo primero que podemos ver es que tenemos 2 paquetes completamente abstractos los cuales los perfilan como fuertes candidatos a estar en la zona de poco uso. Además la solo BusinessLogic respeta la cohesión relacional pero esto no nos preocupa demasiado ya que al favorecer DIP y que nuestro diseño fuera mantenible resulta en un menor acoplamiento entre las clases.

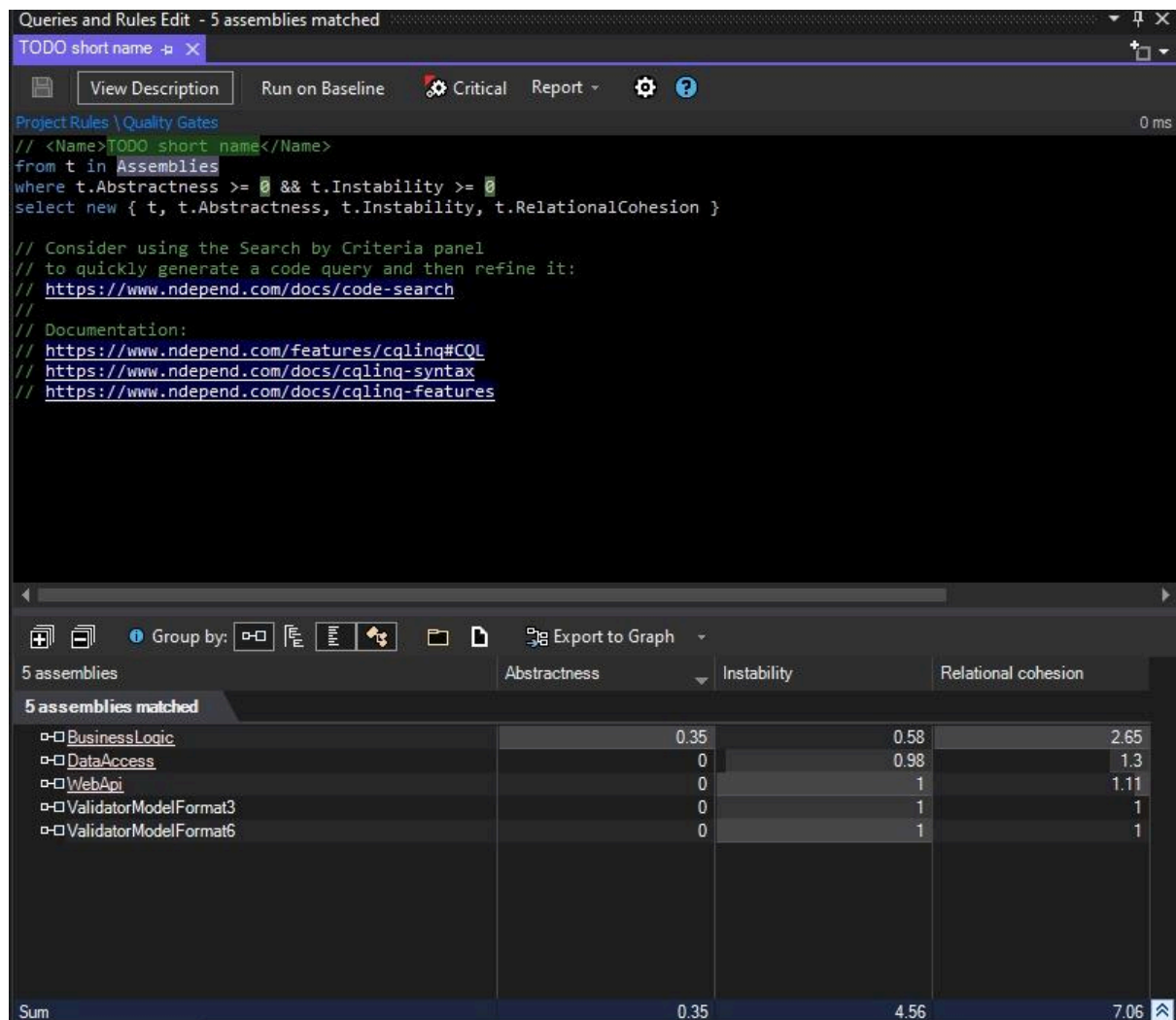
A continuación podemos ver la gráfica entre abstracción e Inestabilidad.



El punto en la zona amarilla es el dominio ya que no tiene clases abstractas y la que esta arriba en la derecha son tanto IbusinessLogic como IDataAccess ya que no tiene mucha utilidad ya que solo ponemos las interfaces en esos paquetes.

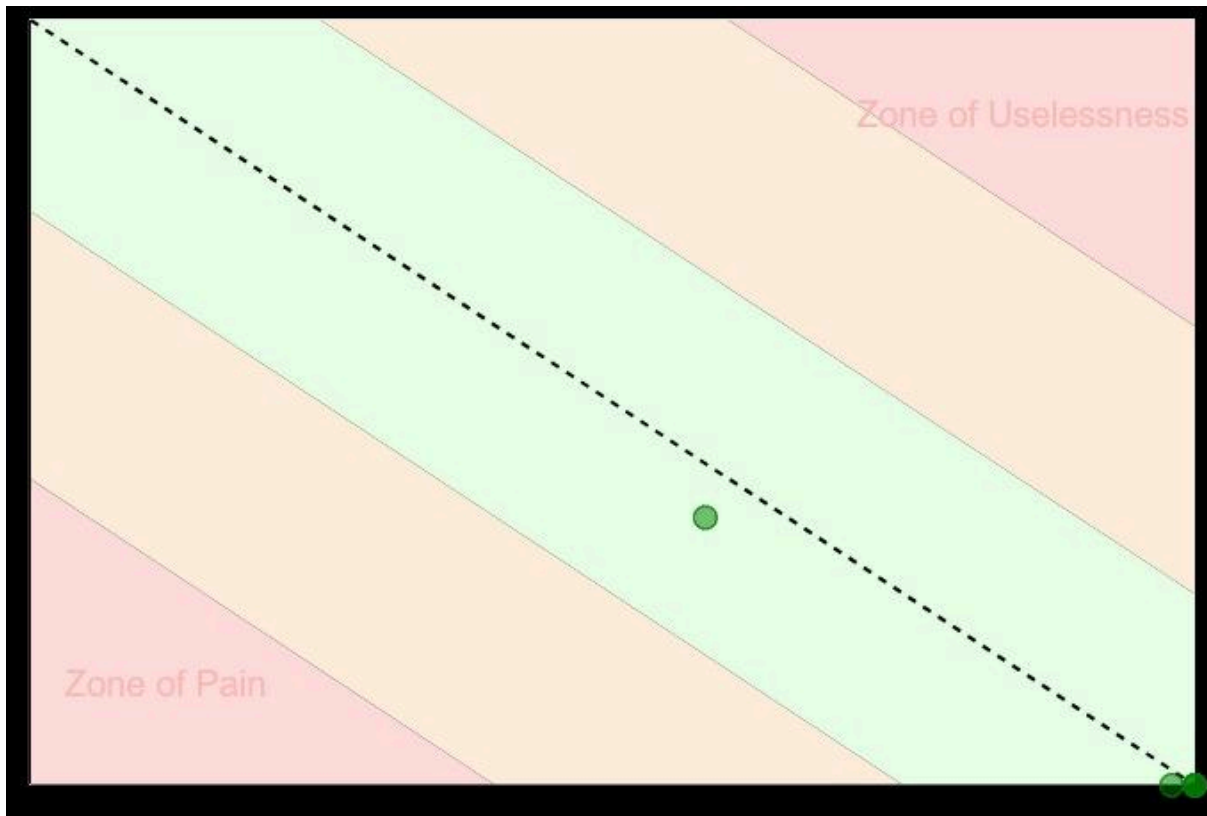
Esto indica que se necesitaba hacer una refactorización de los paquetes que fue la que explicamos al principio. A continuación mostraremos las métricas de esta entrega

Segunda entrega:



Como podemos ver las métricas de cohesión relacional siguen parecidas pero se acercaron un poco más a lo que se conoce como una cohesión relacional saludable. Además ya no tenemos ninguna clase completamente abstracta. Esto nos indica que la API es muy concreta a la vez que inestable pero eso es normal en clases de las capas más “altas”, el paquete bussinessLogic ahora tiene más parte abstracta pero sigue siendo saludable. Mientras que el paquete dataAccess es muy concreto e inestable ya que utilizamos una extensiones de terceros (EntityFramework) para acceder a la base de datos.

A continuación podemos ver que en el gráfico los tres paquetes están en una zona saludable y reagrupamos los paquetes que daban problemas. Esto no solo lo hizo más mantenible si no que lo vimos nosotros mismos al cambiar todo a 3 paquetes solamente se hizo más fácil de trabajar en la solución.



Análisis de las métricas según los principios de paquetes

CPP: Principio de clausura común

Viendo que nuestra cohesión es bastante buena sobre todo en bussinessLogic y DataAcces (2,65 y 1,3 respectivamente podemos decir que las clases de nuestros paquetes están bien agrupados).

CRP: Principio de reutilización

Para que un paquete sea reutilizable tenemos que conseguir que sea relativamente abstracto y estable. Si bien nuestras métricas de abstracciones de WebAPI y BusinessLogic no están mal no los consideraríamos muy reutilizables. Esto no lo logramos de manera muy efectiva sobre todo en la parte de DataAccess en la cual podríamos haber implementado un repositorio genérico para que este sea más reutilizable.

SAP: Principio de abstracción de paquetes.

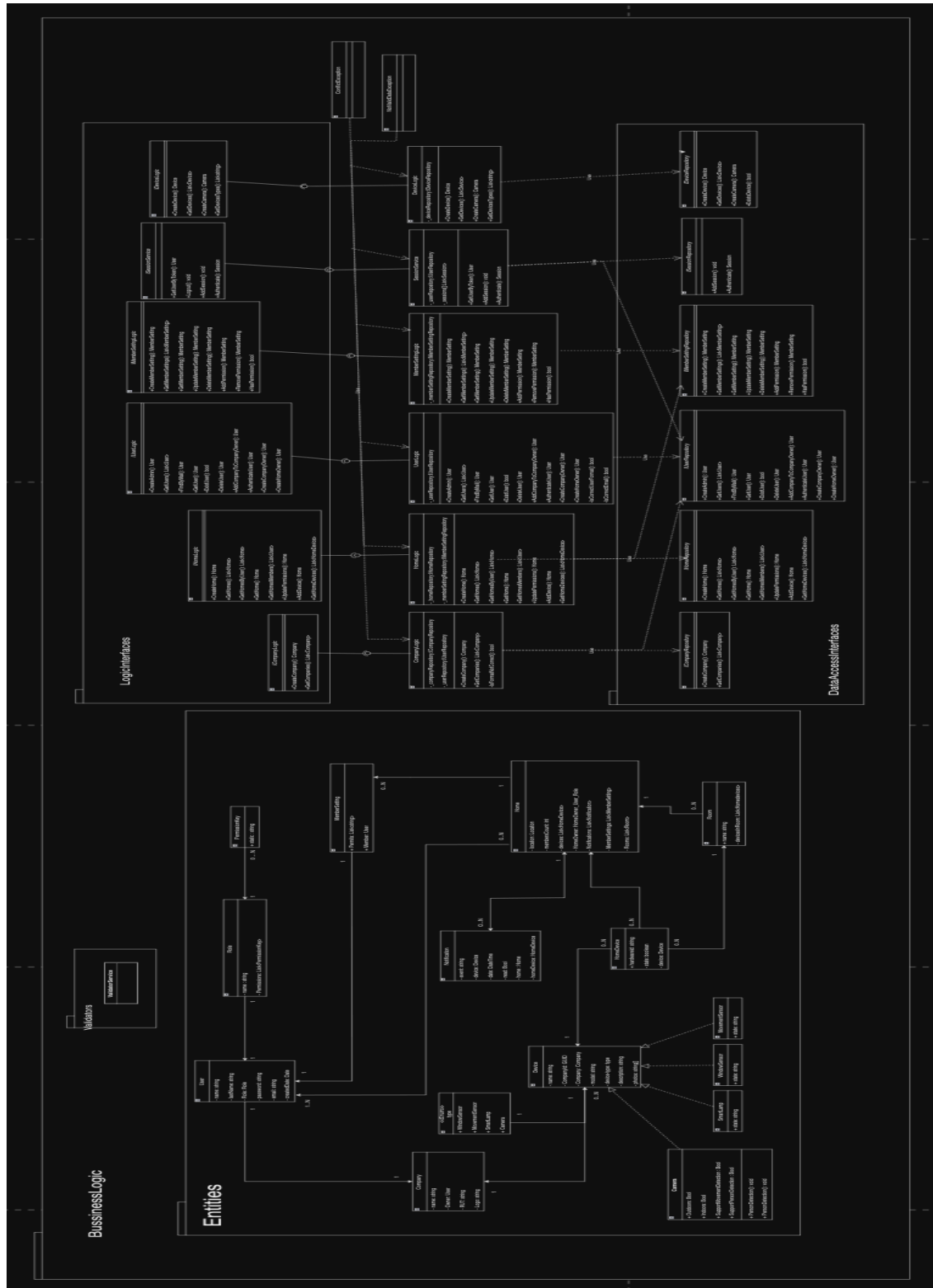
Este principio lo cumplimos ya que nuestro paquete más abstracto (BussinessLogic) también es el más estable.

SDP: Principio de Dependencias Estables.

En principio este principio lo cumplimos de manera correcta ya que nuestro paquete BussinessLogic es más estable que el de WebAPI y a su vez DataAccess usa BussinessLogic que también es más estable que el.

Anexo

Anexo 1: Diagrama de clases de BussinessLogic



Anexo 2: Especificación de la API

Los endpoints encuadrados en azul son los que se modificaron para esta entrega.

Metodo	Recurso	Headers	Endpoint	Parametros	Request Body	Response Body 200	Respuestas	Especificación Del Recurso	Mensaje de Error
POST	Sessions		api/login		{ "email": string, "password": }	{ "token": string, "userRole": string }	201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 400 - Conflict 500 - Internal Server Error	Se utiliza para que un usuario se autentique	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Users		api/admins		{ "email": string, "name": string, "lastname": string, "password": string }		201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 400 - Conflict 500 - Internal Server Error	Se utiliza este recurso para crear nuevos administradores	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
DELETE	Users		api/admins/adminid			{ "email": string, "name": string, "lastname": string, "password": string }	200 - Ok 400 - Bad Request 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Utilizamos este recurso para permitir a los administradores borrar cuentas de administrador	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Users		api/company-owners		{ "email": string, "name": string, "lastname": string, "password": string }		201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 400 - Conflict 500 - Internal Server Error	Utilizamos este recurso para crear una cuenta dueño de empresa	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
GET	Users		api/users	roleId: string (optional) name: string (optional) pageNumber: int (optional) Default: 1 pageSize: int (optional) Default: 10		{ "email": string, "name": string, "lastname": string, "role": string, "CreationDate": string "DDMM/YYYY" }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se Utiliza para que los administradores puedan ver las cuentas de usuario de la plataforma	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string

Metodo	Recurso	Headers	Endpoint	Parametros	Request Body	Response Body 200	Respuestas	Especificación Del Recurso	Mensaje de Error
GET	Users		api/users/userid			User: { "name": string, "lastname": string, "email": string, "role": string, "CreationDate": string "DDMM/YYYY" }	200 - Ok 401 - Unauthorized 500 - Internal Server Error	Se Utiliza para que los administradores puedan ver las cuentas de un usuario de la plataforma	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
GET	Users		api/users/userEmail			User: { "name": string, "lastname": string, "email": string, "role": string, "CreationDate": string "DDMM/YYYY" }	200 - Ok 401 - Unauthorized 500 - Internal Server Error	Se Utiliza para recuperar la cuenta de un usuario por su email	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
DELETE	Users		api/users/userid			{ "email": string, "name": string, "lastname": string, "password": string }	200 - Ok 400 - Bad Request 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Utilizamos este recurso para permitir a los administradores borrar cuentas de usuarios	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
GET	Users		api/users/userid/homes			{ "homes": ["quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string "HomeOwner": string] }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para traer todas las homes del user asociado	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
GET	Users		api/users/notifications			Notifications: [{ "event": string, "timestamp": string, "user": string, "timestamp": string, "CreationDate": string "DDMM/YYYY" }]	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para traer todas las notificaciones del user asociado	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Companies		api/companies		{ "name": string, "url": string, "photo": string }		201 - Created 400 - Bad Request 401 - Unauthorized 400 - Conflict 500 - Internal Server Error	Se utiliza para crear una nueva empresa	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
GET	Companies		api/companies	name: string (optional) OwnerName: string (optional) pageNumber: int (optional) Default: 1 pageSize: int (optional) Default: 10		{ "companies": ["name": string, "OwnerName": string, "OwnerEmail": string, "URL": string,] }	200 - Ok 401 - Unauthorized 500 - Internal Server Error	Se Utiliza para que los administradores puedan ver las empresas de la plataforma	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string

	A	B	C	D	E	F	G	H	I	J
1	Verbo	Recurso	Headers	Endpoint	Parametros	Request Body	Response Body 200	Respuestas	Especificación Del Recurso	Mensajes de Error
17	POST	Homes	Authorization	api/homes		{ "name": string "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }	{ "name": string "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string, "homeOwner": string }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para que un usuario cree una Hogar	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
18	POST	Homes	Authorization	api/homes/rooms		{ "roomName": string }	{ "roomName": string }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para crear una nueva room en una home particular	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
19	PUT	Homes	Authorization	api/homes/homeId		{ "homeName": string }	{ "roomName": string }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para cambiar el nombre room en una home particular	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
20	GET	Homes	Authorization	api/homes			{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string, "homeOwner": string }	201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Se utiliza para conseguir la lista de las Homes	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
21	PUT	Homes	authorization	api/homes/homeId/members/userId		{ "Members": [] "direction": string, "latitude": string, "longitude": string }		200 - Ok 400 - Bad Request 401 - Unauthorized 409 - Conflict 500 - Internal Server Error	Se utiliza para modificar los permisos de un miembro de una Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
22	GET	Homes	authorization	api/homes/homeId/members			{ Users: [{ "name": string, "username": string, "email": string, "createdAt": string, "DDMM/YYYY" }] }	200 - Ok 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Tras la lista de miembros de una Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
23	GET	Homes	Authorization	api/homes/homeId			{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string, "homeOwner": string }	201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Tras los datos de una Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string ➡ Convertir en chips de menú desplegable

Convertir en chips de menú desplegable

Verbo	Recurso	Headers	Endpoint	Parametros	Request Body	Response Body 200	Respuestas	Especificación Del Recurso	Mensajes de Error
GET	Homes	Authorization	api/homes/homeId/devices			{ Devices: [{ "name": string, "model": string, "photo": string, "company": string }] }	201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Tras los Devices asociados a cierta Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	Authorization	api/homes/homeId/devices		{ "name": string, "model": string, "photo": string, "company": string }		201 - Created 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Se utiliza para agregar un Device a una Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
PUT	Homes	authorization	api/homes/homeId		{ "Members": [] "Name": string "direction": string, "latitude": string, "longitude": string }		200 - Ok 400 - Bad Request 401 - Unauthorized 409 - Conflict 500 - Internal Server Error	Se utiliza para agregar Members a la Home	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
PUT	Homes	authorization	api/homes/homeId/rooms/roomId		{ "roomName": string }		200 - Ok 400 - Bad Request 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para cambiar el nombre de una room ya existente	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	authorization	api/homes/homeId/rooms		{ "roomName": string }		200 - Ok 400 - Bad Request 401 - Unauthorized 403 - Forbidden 409 - Conflict 500 - Internal Server Error	Se utiliza para agregar Rooms a una Home especifica	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	Authorization	api/homes/homeId/windowSensor/hardwareId/open		{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }		200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para que un usuario cree una Hogar	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	Authorization	api/homes/homeId/windowSensor/hardwareId/open		{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }		200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para crear una notificación de alerta	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	Authorization	api/homes/homeId/windowSensor/hardwareId/close		{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }		200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para crear una notificación de cerrado	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
POST	Homes	Authorization	api/homes/homeId/camera/hardwareId/person-detected		{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }		200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para que un usuario cree una Hogar	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string

Convertir en chips de menú desplegable

	A	B	C	D	E	F	G	H	I	J
1	Verbo	Recurso	Headers	Endpoint	Parametros	Request Body	Response Body 200	Respuestas	Especificación Del Recurso	Mensajes de Error
33	POST	Homes	Authorization	api/homes/homeId/camera/hardwareId/movement-detected		{ "quantityOfMembers": int, "direction": string, "latitude": string, "longitude": string }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para que un usuario cree una Hogar	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string	
34										
35	GET	Devices	authorization	api/devices	name - string (optional) model - string (optional) DeviceType: int (Optional) 0-Sensor 1-Camera companyName - string (optional) pageNumber: int (optional) Default 1 pageSize: int (optional) Default 10	{ Devices: [{ "name": string, "model": string, "photo": string, "company": string }] }	200 - Ok 401 - Unauthorized 500 - Internal Server Error	Tras una lista de todos los Devices	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string	
36	GET	Devices	authorization	api/devices/types		{ devices-types: [{ "Type": string }] }	200 - Ok 401 - Unauthorized 500 - Internal Server Error	Tras una lista de todos los tipos de Devices	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string	
37	POST	Devices	authorization	api/devices/sensor		{ "name": string, "model": string, "photo": string, "Company": string }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para crear un Device	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string	
38	POST	Devices	authorization	api/devices/cameras		{ "name": string, "model": string, "Photo": string, "CompanyName": string, "Outlook": bool, "Indoor": bool, "SupportMovement": bool "SupportPersonDetection": bool }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se utiliza para crear una Camera	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string	
39	POST	Devices	authorization	api/devices/movementSensor		{ "name": string, "model": string, "Photo": string, "CompanyName": string, }	{ "name": string, "model": string, "Photo": string, "CompanyName": string, }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se Utiliza para crear un sensor de movimiento	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
40	POST	Devices	authorization	api/devices/smartlamp		{ "name": string, "model": string, "Photo": string, "CompanyName": string, }	{ "name": string, "model": string, "Photo": string, "CompanyName": string, }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se Utiliza para crear un sensor de movimiento	Body de Código de Error para cualquier endpoint: innerCode: string code: int message: string
				</						

Convertir en chips de menú desplegable

Metodo	Recurso	Headers	Endpoint	Parameters	Request Body	Response Body 200	Respuestas	Explicación Del Recurso	Mensajes de Error
POST	Devices	authorization	api/devices/martamp		{ "name": string, "brand": string, "Photo": string, "CompanyName": string, }	{ "name": string, "brand": string, "Photo": string, "CompanyName": string, }	200 - Ok 401 - Unauthorized 403 - Forbidden 500 - Internal Server Error	Se Utiliza para crear un sensor de movimiento	Body de Código de Error para cualquier endpoint: errorCode: string code: int message: string
POST	Home-owner		api/home-owner		{ "email": string, "name": string, "lastName": string, "password": string, }		200 - Ok 401 - Unauthorized 500 - Internal Server Error	Utilizamos este recurso para crear una cuenta dueño de Home	Body de Código de Error para cualquier endpoint: errorCode: string code: int message: string
GET	Validators		api/validators/load-validators			{ validators: ["name": string] }	200 - OK 400 - Bad Request 500 - Internal Error	Se utiliza para cargar los validadores que hay	Body de Código de Error para cualquier endpoint: errorCode: string code: int message: string
POST	Import Devices	authorization	api/import-devices		{ "CompanyName": string, "AssemblyPath": string, }	{ "Devices Imported" }	201 - Created 400 - Bad Request 500 - Internal Error	Se utiliza para importar dispositivos	Body de Código de Error para cualquier endpoint: errorCode: string code: int message: string

Convertir en chips de menú desplegable

Anexo 3: evidencia de Code Coverage

Symbol	Coverage (%)	Uncovered/Total Stmts
Total	91%	182/1950
Test	100%	0/78
BusinessLogic	91%	57/668
BusinessLogic	91%	57/668
ConflictException	100%	0/3
EmptyException	100%	0/3
MemberSettingLogic	100%	0/32
NotValidDataException	100%	0/3
HomeLogic	94%	10/157
Entities	93%	13/194
UserLogic	90%	10/98
CompanyLogic	89%	4/37
DeviceLogic	88%	10/80
Validators	86%	3/22
SessionService	82%	7/39
DataAccess	90%	47/454
DataAccess	90%	47/454
NotificationRepository	97%	2/64
Repositories	88%	45/390
WebApi	90%	78/750
Models	92%	2/26
WebApi	90%	76/724